

Proyecto de Curso

A. Contexto general

En 2026, la Copa Mundial de la FIFA se celebrará por primera vez con un formato ampliado y una organización multinacional. El evento será distribuido entre tres países anfitriones y múltiples ciudades sede, lo que agrega una complejidad logística para aficionados, operadores, marcas, medios y aliados comerciales. En paralelo, la audiencia digital espera una experiencia confiable: información oportuna, notificaciones relevantes, contenido coherente y una alta disponibilidad dado que el evento es seguido por millones de personas.

En este escenario surge Mundial 2026 Hub, una plataforma web y móvil orientada a la experiencia digital del torneo. La idea no es competir con canales oficiales, sino cubrir un problema recurrente en eventos de gran escala: el usuario final se mueve entre muchas fuentes, horarios, cambios de última hora, y canales de compra de entradas y merchandising; mientras que los equipos operativos requieren herramientas para publicar información, gestionar incidentes y mantener trazabilidad.

El sistema propone un núcleo informativo y operacional con cuatro ejes: (1) seguimiento de partidos y estadísticas, (2) agenda personalizada y planificación por ciudades/estadios, (3) notificaciones multicanal y comunicación operativa, y (4) flujos de entradas y atención de solicitudes.

Propósito del producto

Mundial 2026 Hub busca ofrecer a cada aficionado una experiencia enfocada en sus preferencias: sus selecciones favoritas, sedes de interés, y un calendario personal que se adapta a cambios. Al mismo tiempo, habilita un “backoffice” para operadores (contenido, soporte y compliance) que necesitan publicar información, supervisar señales de fraude o abuso, y responder a solicitudes con evidencia.

El valor diferenciador, y el hilo conductor del proyecto, es la transparencia y trazabilidad: el sistema registra eventos relevantes (cambios en calendario, notificaciones emitidas, compras simuladas, transferencias, reembolsos, decisiones antifraude) de forma auditable.

Interesados y expectativas

Aficionados locales quieren entrar y “ver lo que importa ya”: próximos partidos, resultados, alertas y contexto. No necesariamente entienden zonas horarias, fases o cambios operativos; esperan que la plataforma sea clara y estable.

Aficionados viajeros viven un problema distinto: necesitan una agenda por ciudad/estadio, recomendaciones logísticas (mapa, tiempos estimados, recordatorios), y alertas confiables para no perder conexiones ni accesos. Su tolerancia al error es baja: un cambio de horario o sede mal comunicado significa gastos reales.

Familias y grupos quieren coordinar: compartir agendas, repartir responsabilidades (quién compra, quién recibe notificaciones), y en algunos casos transferir entradas dentro del grupo sin caer en fricción o riesgo.

Operadores del evento (contenido y operaciones) necesitan un canal para publicar cambios controlados, evitar rumores, y activar comunicados masivos con segmentación (por partido, ciudad o estadio).

Soporte al usuario enfrenta casos típicos: “no me llegó la confirmación”, “mi entrada expiró”, “no puedo acceder”, “me equivoqué de partido”. Para resolver con calidad y rapidez, soporte necesita contexto, historial y eventos de auditoría.

Legal / compliance busca minimizar riesgo: uso indebido de cuentas, patrones anómalos de compra/transferencia, reclamos masivos. Su expectativa es que el sistema pueda demostrar qué pasó y por qué.

Aliados comerciales (ticketing, pagos, mensajería, datos deportivos) esperan integraciones responsables: límites de consumo, seguridad de tokens, manejo correcto de errores y reintentos.

Alcance funcional (MVP y crecimiento)

Se espera un MVP suficiente para validación operativa y de usuarios, y luego se espera poder aumentar las funcionalidades e integraciones en iteraciones futuras.

En el MVP, el usuario se registra/inicia sesión, configura preferencias, obtiene un calendario personal y recibe notificaciones básicas. Adicionalmente permite la conformación de comunidades o grupos de aficionados, puede ser la misma familia de los usuarios y/o entre amigos, para realizar las llamadas “pollas futboleras”, mediante las cuales se acumularan puntos y la plataforma dará premios digitales al finalizar el evento.

En el módulo de pollas, los aficionados suelen crear grupos con amigos, familia o compañeros de trabajo para “apostarle” a resultados. Se modela como un juego social de predicciones por puntos, con reglas claras, cierre de pronósticos antes del partido, cálculo automático de puntajes, ranking y evidencia auditada de cada pronóstico. El sistema calcula y consolida resultados cuando el partido finaliza y publica un Ranking.

La segunda es el módulo de álbum digital. En los mundiales, el álbum de láminas es un fenómeno cultural: la gente colecciona, intercambia repetidas y completa

páginas por selecciones y figuras. Para 2026 se ha comunicado que el álbum oficial tendrá una escala mayor que ediciones previas, lo cual es un catalizador natural para una experiencia digital paralela. Los patrocinadores de la iniciativa proponen un álbum digital donde el usuario obtiene “paquetes” (por acciones dentro de la app, códigos promocionales, o puntos acumulados para los ganadores de las pollas, cada usuario podrá organizar su colección, identificar repetidas, intercambiar con su comunidad.

Un partido representa un encuentro con fecha/hora, estadio, selecciones, fase y estado (programado/en juego/finalizado). Un Usuario tiene un perfil, preferencias (equipos, sedes, ciudad), y configuración de notificaciones. Una Entrada es un activo digital asociado a un partido y a un titular; su ciclo de vida se controla por estados y reglas. Un Evento es cualquier ocurrencia relevante (gol, tarjeta, cambio, actualización de horario), pero también acciones del sistema (notificación enviada, reserva expirada, bloqueo antifraude).

En el módulo de colección, el Álbum es una estructura de páginas y secciones (por selección, estadio o categoría). Una Lámina (sticker) es un elemento coleccionable con rareza y metadatos (jugador, equipo, categoría). Un Paquete es un conjunto de láminas que se abre bajo reglas de asignación; si aparecen repetidas, se convierten en “moneda de intercambio” o se habilita el intercambio directo entre usuarios.

Onboarding, preferencias y agenda personal

El recorrido inicia cuando el aficionado crea su cuenta. El sistema le propone un asistente de configuración: elegir selecciones favoritas, ciudades o estadios, y canales de notificación. A partir de esto, se construye una agenda personal y un “feed” que prioriza lo relevante. Si el usuario viaja, el sistema lo acompaña con recordatorios contextuales (zona horaria, ventanas de acceso y alertas antes del partido).

Ingesta de datos de partidos y publicación al usuario

La plataforma consume datos de proveedores externos sobre equipos, partidos, estadios, resultados y eventos. Cuando la fuente falla o responde lento, el sistema debe degradar con gracia: mostrar el último dato confirmado, etiquetar “actualización pendiente” y evitar que una falla de terceros tumbé toda la experiencia.

Notificaciones operativas y personalizadas

Cuando inicia un partido, ocurre un gol o se confirma un cambio, el sistema evalúa reglas de suscripción: quién debe ser notificado, por qué canal y con qué prioridad.

En paralelo, el backoffice permite a un operador activar notificaciones masivas o segmentadas. Para el usuario, la notificación debe ser oportuna y clara; para el operador, debe haber evidencia (qué se envió, a quién, cuando y por qué medio).

Flujo de entrada (reserva, confirmación y expiración)

El sistema ofrece un flujo de gestión de entradas con un principio central: una entrada solo puede estar en un estado a la vez, y cada transición deja trazabilidad.

Los pagos deberán realizarse de forma simulada con wiremock (estableciendo un contrato – API – detallado y con reglas para su consumo) o puede hacer uso de ambientes de prueba de Stripe o MercadoPago. Recuerde que las reservas pueden tener un tiempo limitado.

Transferencia y reembolso con controles antifraude

El usuario puede transferir una entrada dentro de reglas predefinidas. La transferencia requiere validación, y el sistema registra no repudio: quién transfiere, quién recibe y cuándo. Para reembolsos, el sistema aplica reglas y estados, genera evidencia y notifica. Si se detectan patrones anómalos, se limita temporalmente la cuenta y se genera un caso para revisión.

Atención de casos e investigación

Soporte recibe una solicitud, consulta la línea de tiempo de eventos, valida evidencias (notificaciones, expiraciones, bloqueos, pagos de prueba) y ejecuta acciones permitidas (reintentar notificación, reactivar reserva si aplica, abrir investigación). Compliance puede auditar transacciones agregadas sin exponer información sensible.

Pollas y publicación de resultados

Un usuario crea una polla para su comunidad. Se otorgan puntos por acertar ganador/empate, más puntos por acertar marcador exacto. La plataforma genera un enlace o código de invitación, y los miembros se unen. Antes de cada partido, el sistema recuerda el cierre de pronósticos y bloquea cambios minutos antes del inicio para evitar ventajas. Cuando el partido termina, el sistema consolida el resultado oficial, calcula puntajes y actualiza el ranking.

Álbum digital, apertura de paquetes e intercambio de repetidas

El usuario quiere “completar su álbum” durante el torneo. La plataforma le entrega paquetes digitales bajo reglas de negocio (por actividad: iniciar sesión diario, completar predicción, seguir partidos, o por códigos promocionales). Al abrir el paquete, el usuario ve qué láminas obtuvo; las nuevas se pegan al álbum y las repetidas se separan. El sistema ofrece dos rutas: intercambio directo (usuario a usuario) con confirmación mutua, o intercambio mediado por “moneda” basada en repetidas. Para evitar abuso, se establecen límites de intercambio y se registran transacciones como eventos auditables.

Reglas de negocio y estados (entradas)

La entrada a un estadio recorre un ciclo de vida controlado: *Disponible* → *Reservada* → *Pagada* → (*Transferida* | *Reembolsada* | *Expirada*). La reserva tiene un tiempo máximo de retención (TTL) y, si no se confirma dentro de la ventana, expira y libera cupos. La plataforma aplica límites parametrizables por usuario (compras/transferencias por día), y registra toda transición con un identificador de correlación.

Integraciones externas

El proyecto prioriza integraciones que existan y tengan planes gratuitos o entornos de prueba. A continuación se proponen opciones reales que pueden ser evaluadas por el equipo para escoger una “combinación” viable.

Necesidad	Alternativas de integración	Observaciones
Datos de partidos, fixtures, tablas	football-data.org (plan gratuito), API-FOOTBALL (plan gratuito), TheSportsDB (API gratuita)	En planes gratis pueden existir límites de cuota o retraso en marcadores.
Notificaciones (web/app)	push Firebase Cloud Messaging (FCM)	Adecuado para notificaciones a clientes web/móvil. Útil para evaluar colas, reintentos y entrega.
Email transaccional (opcional)	SendGrid (trial / límites)	Útil para confirmaciones y trazabilidad de entrega; debe respetar límites.
Pagos en modo prueba	Stripe (test mode sandboxes)	Permite simular cobros, reembolsos y disputas sin dinero real.
Mapas y geocodificación (sedes/ciudades)	OpenStreetMap / Nominatim	Útil para búsquedas geográficas; requiere respetar políticas de uso y límites.

Recuerde que puede generar sus propios APIs usando Wiremock Cloud, la restricción es clara y debe haber un contrato y documentación de uso muy bien definidos.

Restricciones de negocio

El proyecto se enmarca como un producto académico. Por ello, se establecen restricciones para evitar riesgos: el sistema no debe promover reventa real de entradas ni operar como intermediario comercial. Cuando se incluyan flujos de

compra, se debe hacer en modo simulado o sandbox. La plataforma debe respetar privacidad: no se expone información personal a terceros.

En el caso de las pollas, el sistema se plantea como un juego social por puntos. No se implementa juego con dinero real ni se gestiona dinero entre usuarios.

En el caso del álbum, no se vende contenido; se simula la obtención de paquetes y el intercambio, y se controlan límites para prevenir abuso (por ejemplo, automatización masiva de intercambios). El sistema debe tener reglas claras de uso aceptable. No use imágenes de personas reales, puede usar “avatars”

Además, el sistema debe operar bajo el principio de “información confiable”: si un dato no está confirmado por el proveedor, se marca como provisional. En caso de conflicto de fuentes, se prioriza una fuente configurada como “principal” y se registra discrepancia como evento interno.

Restricciones tecnológicas

El patrocinador del proyecto recomienda una arquitectura orientada a servicios con separación clara entre presentación, lógica de negocio y persistencia. El backend expone una API REST y/o eventos, y las integraciones se encapsulan en adaptadores. Se recomienda instrumentación de observabilidad desde el inicio (logs estructurados con correlación, métricas y trazas).

Dado que el consumo de datos puede tener límites, se recomienda incluir caché y estrategias de actualización (polling, webhooks si existen, o colas internas). El sistema debe estar preparado para cambiar de proveedor de datos sin reescribir toda la lógica.

B. Fases del Proyecto

Las fases que se recomiendan para este proyecto son cuatro, a saber:

1. Análisis de Requerimientos:

- Identificar los procesos de negocio.
- Identificar interesados.
- Identificar o proponer una EDT o WBS de alto nivel.
- Identificar y priorizar los requerimientos funcionales y no funcionales.
- Modelizar¹ los casos de uso.

¹ La traducción más acertada de “modelling” del inglés al castellano es “Modelización” y no “Modelación”. No son sinónimos en castellano.

- Redactar las historias de usuario según las necesidades detectadas.
 1. Seguramente podrá encontrar: epics, features e historias de usuario.
 2. Puede consultar <https://www.atlassian.com/agile/project-management/epics-stories-themes>
- Estimar las historias de usuario. Recuerde que la estimación de cada historia es el puntaje asignado en la sesión de estimación por el equipo, no por un integrante individualmente. Debe dejar registro de una sesión de estimación en video de no más de 10 min. Debe entregar el link al video en youtube. Debe hacer uso de la herramienta planning Poker.
- Planear los *sprints* o iteraciones
 1. Cronograma de alto nivel
 1. Incluya los releases (entregables del producto de software) que realizará de acuerdo con el MVP esperado.
 2. Tenga en cuenta el MVP al momento de planear y priorizar sus sprints e historias de usuario.
- Riesgos de alto nivel

2. Diseño:

- Prototipo de pantallas del sistema de acuerdo con el proceso o los procesos de negocio, tenga en cuenta el MVP.
- Documentar la arquitectura utilizando diagramas UML y Modelo C4.
- Modelo de datos (Diseño lógico y Diseño Físico de la base de datos o bases de datos)

Modelización:

Modelación hace referencia a la acción y efecto de modelizar (RAE) que por su parte significa **construir el modelo de algo**.

Modelación hace referencia a la acción y efecto de modelar (RAE) que por su parte **significa formar, configurar o conformar algo**.

Para este contexto, la connotación apropiada es la de “*construir el modelo* de algo” a través de los casos de uso y de los distintos objetos tecnológicos asociados. Lamentablemente, existe una propensión a traducir directamente “modelling” como “modelación” lo cual, en general, no es apropiado.

3. Implementación:

- Dividir las historias de usuario en tareas.
- Asignar las historias de usuario a los integrantes del equipo y planear los sprints de desarrollo. Cabe anotar que al finalizar cada Sprint se debe entregar valor al usuario (Tenga en cuenta el MVP). El grupo debe pensar en un objetivo por sprint, seleccionar las historias a implementar de acuerdo a ese objetivo y demostrarlo en una sesión de review.
- Implementar el sistema usando buenas prácticas de programación, respetando la separación de responsabilidades y el estilo de arquitectura de acuerdo con los diseños realizados.
- DevOps y Despliegue:
 - Configurar un pipeline de integración y despliegue continuo (CI/CD).
 - Integrar un sistema de *logs* centralizado como Splunk o Elasticsearch
 - Repositorio Git
 - Automatización con GitHub Actions
 - Contenerización (Docker) (Opcional)
 - Automatización despliegue (Opcional)

4. Pruebas y Validación

- Diseñar casos de prueba para validar las funcionalidades implementadas: Pruebas Unitarias y Pruebas de Integración.
- Validar la trazabilidad de eventos en el sistema, como operaciones exitosas, fallos, intentos de autenticación, y accesos no autorizados,
- Corregir errores y ajustar el sistema según los resultados de las pruebas. Debe quedar documentado como parte del proceso de Ingeniería de Software.

Productos Esperados

Documentación Completa:

- De alto nivel: Cronograma, Riesgos, EDT / WBS.

- Requerimientos funcionales y no funcionales.
- Diagramas UML y Modelo C4
 1. Procesos (BPMN)
 2. Contexto (Incluya análisis desde la perspectiva de sistema de información de la Facultad: Software, Personas, Información, Infraestructura, Procesos de negocio)
 3. Casos de uso
 4. Clases
 5. Componentes
 6. Secuencia, al menos para 3 flujos
 7. Despliegue
- Historias de usuario con criterios de aceptación y evidencia de pruebas realizadas (JIRA).
 1. Epícs, Features
 2. Product Backlog
 3. Sprint backlog, evidencia de planeación y estimación de historias (por cada sprint o interacción)
 4. Las historias de usuario deben seguir el formato especificado. Título, Descripción, Criterios de Aceptación y Estimación.
 5. Las historias de usuario deben estar creadas, documentadas y estimadas en Jira.
 1. Para la priorización de historias de usuario use la técnica Moscow
 2. (<https://community.atlassian.com/t5/App-Central-articles/Understanding-the-MoSCoW-prioritization-How-to-implement-it-into/ba-p/2463999>)
- Evidencia de las pruebas realizadas, análisis, diseño y validación de los atributos de calidad identificados en el caso.
- Documentación del pipeline de CI/CD. Debe incluir todo lo necesario para replicar la configuración en caso de ser necesario.

2. Producto Funcional:

- De acuerdo a los requerimientos, diseños y priorización.
- MVP (Mínimo Producto Viable):
 1. Registro e inicio de sesión
 2. Gestión de Perfil
 3. Pollas
 1. Incluyendo Ranking y acumulación de puntos
 4. Album Digital
 1. Incluyendo intercambios
 5. Monitoreo y trazabilidad con Logs

3. DevOps:

- Pipeline de CI/CD implementado.
- Configuración de herramientas adicionales como SonarCloud para análisis estático de calidad del código.
- Uso de archivos de configuración para manejar el despliegue en diferentes entornos (desarrollo, prueba y producción) y versiones.

4. Informe Final:

Debe contener lo descrito en los puntos anteriores, evidencia del uso de las herramientas seleccionadas para el desarrollo del caso, y la justificación de las decisiones a nivel de diseño y arquitectura de la solución. Adicionalmente:

- Reflexión sobre las lecciones aprendidas.
- Evaluación del cumplimiento de los objetivos iniciales y MVP (Mínimo Producto Viable)
- Qué aportes haría al sistema para mejorar su funcionamiento? Qué considera hace falta?

A tener en cuenta:

Teniendo en cuenta la complejidad del caso, los siguientes puntos se consideran opcionales, en caso de ser entregados y sustentados por los grupos de trabajo tendrán una bonificación adicional:

- Pruebas de Integración
- Contenerización² (*Docker*)
- Autenticación Multifactor
- Pull Request en GitHub integrados con Sonar

Tenga en cuenta que el docente podrá solicitar en cualquier momento revisiones al estado de avance del proyecto, para lo cual deberá, obligatoriamente, ser usado el board de Jira. El Board será la única fuente de verdad sobre los avances del proyecto, una entrega sin board, sin proceso no será válida.

Tenga en cuenta que la nota del proyecto se basa en entregables (producto y proceso), sustentación (producto y proceso) y funcionalidad.

Uso de Herramientas

Para el desarrollo del proyecto se espera el uso de las herramientas que soportan un proceso de Ingeniería de Software y de gestión de proyectos entre ellas:

- JIRA (Sprints, historias de usuario, objetivos de sprint)
 - <https://www.atlassian.com/software/jira?referer=jira.com>
 - <https://planitpoker.com/> (estimaciones)
- Draw.io, cacao.com, cloudcraft (modelos UML y C4), excalidraw
- Figma (prototipos, usabilidad y experiencia de usuario)
- GitHub - GitHub Actions / Jenkins
- Git – GitHub (Repositorio de Código)
 - Rama main o principal: Contendrá el código funcional que se ha verificado y probado y puede ser enviado a producción.
 - Ramas feature: Contendrá el código que se está trabajando actualmente para desarrollar 1 historia de usuario.
- SonarCloud (análisis de código estático)
- Stripe, Stripe Billing (<https://docs.stripe.com/testing>). No debe ser incluido en las pruebas de rendimiento con JMeter. Tenga en cuenta que Stripe ofrece un espacio para pruebas de la integración con su aplicativo. Use los datos que Stripe tiene para pruebas:

² Quizás la palabra no existe en castellano, pero es usada en los contextos de Ingeniería de Software.

- <https://docs.stripe.com/testing#cards>
- APIs de información sobre partidos, equipos, estados, etc. En caso de no seleccionar una API existente debe generar las diferentes APIs a usar en Wiremock Cloud, debe entregar documentación de la API donde se observe un contrato claro para su uso: request, response, objetos involucrados, posibles códigos de respuesta.
- Puede usar Postman o Insomnia para aprender, probar, verificar usar las APIs que de acuerdo a su análisis tengan las funcionalidades disponibles en para suplir los requerimientos y necesidades del caso. Para este proceso es clave el análisis realizado y la identificación de procesos de negocio y requerimientos.
- Splunk, ElasticSearch. El equipo de trabajo debe investigar y seleccionar la herramienta. Recuerde que debe definir la estructura de logs adecuada que facilite la indexación para la búsqueda de información.
- Java, Python, Go, .NET o cualquier otro lenguaje es permitido para el desarrollo del backend del proyecto. El grupo de trabajo deberá escoger de acuerdo con las habilidades de los integrantes. El uso de frameworks como Spring para Java es totalmente válido.
- JSF, Angular, React, Vue, o cualquier otro framework es permitido para el desarrollo del frontend del proyecto. También es válido el uso de plantillas / templates disponibles o componentes ya existentes en el mercado como Angular/React Material, PrimeNG, entre otros. El grupo de trabajo de escoger de acuerdo con las habilidades de los integrantes.
- Para los modelos UML y C4 se permite el uso de la herramienta con la que tenga más familiaridad el grupo de trabajo, se sugiere: draw.io, caco.com, cloudcraft. El grupo de trabajo podrá escoger.
- Siempre deberá respetarse siempre las convenciones de nombramiento y la notación ya sea UML, BPMN o C4.